

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

Jnana Sangama, Belgaum-590018



Mini Project Report on

“Gesture Controlled System”

Submitted in partial fulfillment of the requirements for the
Second Semester of the Bachelor of Engineering Degree, towards the completion of the
Mini Project under the **Interdisciplinary Project**, Department of Basic Sciences.

by

SN	Name	USN/Roll number
1	NISHANT SINGH	1CR25IS108
2	NITHIN H	1CR25IS109
3	SMRITI	1CR25AI114
4	SNEHIN REDDY NASANAKOTA	1CR25AI115

Under the Guidance of
Dr.Nidhi Joshi



CMR INSTITUTE OF TECHNOLOGY

#132, AECS LAYOUT, IT PARK ROAD, KUNDALAHALLI, BANGALORE-560037

DEPARTMENT OF BASIC SCIENCES



CERTIFICATE

This is to certify that the mini project entitled "**Gesture Control System**" has been successfully carried out by Nishant Singh(1CR25IS108), Nithin H(1CR25IS109), Smriti(1CR25AI114), Snehin Reddy Nasanakota (1CR25AI115), and bonafide students of **CMR Institute of Technology**.

The project is submitted in partial fulfillment of the requirements for the Second Semester of the Bachelor of Engineering Degree, towards the completion of the Mini Project under the **Interdisciplinary Project, Department of Basic Sciences**.

It is further certified that all corrections and suggestions indicated during the Internal Assessment have been duly incorporated in the project report submitted to the departmental library. This File Structures mini project report has been reviewed and approved as it satisfies the academic requirements prescribed for the said degree.

Signature of Guide

Dr.Nidhi Joshi

Dept. of ISE, CMRIT

Signature of HOD

Dr. Fazlur Rahaman

HoD, Dept. of Chemistry, CMRIT

External Viva

Name of the examiners

Signature with date

1.

2.

ACKNOWLEDGEMENT

I sincerely express my gratitude to **Dr. Sanjay Jain**, Principal, CMR Institute of Technology, Bangalore, for providing a supportive academic environment.

I extend my thanks to **Dr. Fazlur Rahaman**, HoD, Dept. of Chemistry, CMRIT for his valuable guidance and support.

I am especially grateful to my internal guide, **Dr. Nidhi Joshi**, Department of ISE Engineering, for her/his constant encouragement and guidance throughout this project.

I also thank all the faculty members, non-teaching staff, and others who contributed directly or indirectly to the successful completion of this work.

Nishant Singh (1CR25IS108)

Nithin H (1CR25IS109)

Smriti (1CR25AI114)

Snehin Reddy Nasanakota (1CR25AI115)

Abstract

The project titled Gesture focuses on the development of a real-time sign language recognition system designed to improve communication accessibility for hearing- and speech-impaired individuals. The primary objective of the project is to recognize commonly used hand gestures and translate them into readable text in multiple languages. The system uses computer vision techniques to capture hand movements through a standard camera and applies hand landmark detection to accurately identify key hand positions. These landmarks are processed using a machine learning-based classification model to recognize predefined gestures such as greetings and basic responses. To enhance accessibility, the recognized gestures are displayed in different languages, including English, Hindi, and Kannada, using appropriate Unicode text rendering. The system emphasizes simplicity, efficiency, and real-time performance, allowing it to run on standard hardware without the need for specialized equipment.

The outcome of the project demonstrates that accurate and responsive gesture recognition can be achieved using lightweight methods and a clean interface, making the system suitable for assistive communication, educational applications, and further extension to more advanced gesture-based interaction systems.

Keywords:

Sign language recognition, hand landmark detection, computer vision, machine learning, accessibility technology

TABLE OF CONTENTS

Title	Page No
-------	---------

Certificate	i
-------------------	---

Acknowledgement.....	iii
----------------------	-----

Abstract	iv
----------------	----

Chapter 1

INTRODUCTION

1.1 Brief History of Sign Language Recognition System.....	1
--	---

1.2 Modern Sign Language Recognition Systems	1
--	---

Chapter 2

ABOUT THE PROJECT

2.1 Problem Statement	2
-----------------------------	---

2.2 Description of the Project	2
--------------------------------------	---

2.3 Challenge Statement	2
-------------------------------	---

Chapter 3

DESIGN THINKING

System Architecture of the Gesture Recognition System.....	3
--	---

3.1 Working Procedure (Flow Chart with Description)	3
---	---

3.2 Prototype Description	4
---------------------------------	---

Chapter 4**IMPLEMENTATION**

4.1 Technologies Used.....	5
4.2 Modules Description	8
4.3 Language Translation and Text Rendering.....	13

Chapter 5**RESULTS AND ANALYSIS**

5.1 Experimental Results	14
5.2 User Feedback and Evaluation.....	15

Chapter 6**CONCLUSION AND FUTURE WORK**

6.1 References.....	16
---------------------	----

Chapter 1

INTRODUCTION

Sign language interpretation plays a crucial role in enabling effective communication for hearing- and speech-impaired individuals. Conventional communication methods often rely on human interpreters, which may not always be accessible, affordable, or available in real time. This limitation creates communication barriers in educational, public, and social environments. The objective of this project is to develop a simple, accurate, and real-time sign language recognition system that can translate hand gestures into readable text. The system utilizes computer vision techniques, hand landmark detection, and machine learning algorithms to identify and classify commonly used gestures. By supporting multiple languages and operating on standard hardware, the proposed system aims to provide an accessible and practical solution for everyday communication needs.

1.1 Brief History of Sign Language Recognition Systems

Sign language has traditionally been interpreted through direct human interaction, requiring trained interpreters for effective communication. Early technological solutions relied on glove-based devices and manual input systems, which were costly and inconvenient. With the advancement of image processing, camera-based methods emerged to detect hand movements, though initial systems faced limitations in accuracy and processing speed.

1.2 Modern Sign Language Recognition Systems

Modern systems use computer vision and machine learning techniques to recognize hand gestures in real time. Hand landmark detection enables accurate tracking without wearable devices, while efficient models allow deployment on standard hardware. These advancements have improved accessibility, reliability, and practical usability in real-world applications.

Chapter 2

PROBLEM STATEMENT

2.1 Description

The proposed project, Gesture, aims to develop a real-time sign language recognition system using computer vision and machine learning techniques. The system captures hand movements through a standard camera and processes the visual input using hand landmark detection to accurately identify key points of the hand and fingers. These landmarks are converted into numerical features, which are then analyzed by a trained machine learning classification model to recognize predefined sign language gestures. The system focuses on commonly used gestures to ensure reliability and ease of use. To enhance accessibility, the recognized gestures are displayed as text in multiple languages, including English, Hindi, Gujarati, Spanish, Telugu and Kannada, using proper Unicode rendering techniques and also understanding the alphabets. The application is designed with a clean and minimal interface, allowing it to operate efficiently on standard hardware without requiring additional sensors or wearable devices. This approach ensures portability, affordability, and ease of deployment across different environments.

2.2 Challenge Statement

Developing an effective sign language recognition system presents several technical and practical challenges. Accurate gesture recognition must be achieved despite variations in hand size, shape, orientation, and movement patterns among different users. Changes in lighting conditions, background noise, and camera positioning can also affect system performance. Ensuring real-time processing while maintaining high accuracy is another significant challenge, particularly when running the system on standard hardware with limited computational resources. Additionally, implementing multilingual text output introduces challenges related to font compatibility, Unicode rendering, and consistent readability across languages. Addressing these challenges is essential to creating a robust, scalable, and user-friendly system capable of supporting inclusive communication in real-world scenarios.

Chapter 3

DESIGN THINKING PROCESS

3.1 Design Structure

a) Empathize: Interactions with students, faculty members, and potential users highlighted communication difficulties faced by hearing- and speech-impaired individuals in everyday situations. Users emphasized the need for a simple system that works in real time without requiring special equipment.

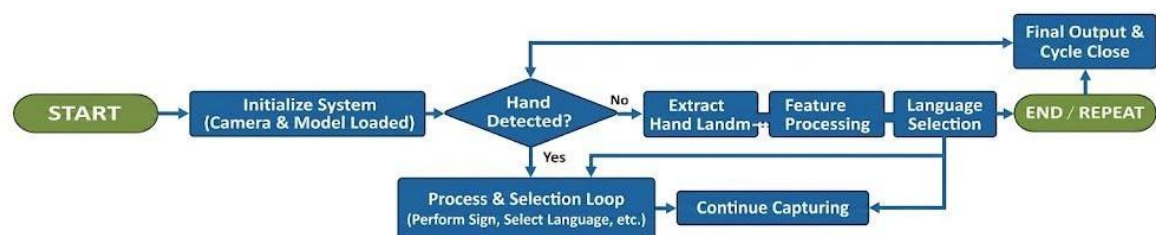
b) Define: Key needs identified included accurate gesture recognition, minimal processing delay, multi-language text output, and ease of use on standard devices without additional hardware.

c) Ideate: Multiple solution approaches were explored, including glove-based systems, depth-camera setups, and vision-based models. The final solution selected was a camera-based sign language recognition system using hand landmark detection and machine learning.

d) Prototype: A functional software prototype was developed using computer vision techniques to capture hand movements, extract landmarks, classify gestures, and display translated text in multiple languages.

e) Test: Testing demonstrated reliable real-time performance with accurate recognition of predefined gestures and successful multilingual text display under varying lighting and user conditions.

3.2 Methodology



3.3 Prototype Description

The prototype of the Gesture system is a software-based sign language recognition application that operates in real time using a standard webcam. It captures hand movements, extracts hand landmarks through computer vision techniques, and classifies gestures using a trained machine learning model. The recognized gestures are displayed as text in multiple languages, including English, Hindi, and Kannada. The prototype is designed with a simple interface and runs efficiently on standard hardware without requiring additional devices, demonstrating the practicality of the proposed approach.

3.3.1 Materials Used

- a) Personal computer or laptop
- b) Webcam or built-in camera
- c) Python programming language
- d) OpenCV library for video processing
- e) MediaPipe framework for hand landmark detection
- f) Machine learning classification model
- g) PIL (Python Imaging Library) for multilingual text rendering h) Trained gesture dataset.

3.3.2 System Design

- Camera Input Module
Captures real-time video frames of hand movements using a standard webcam.
- Hand Detection Module
Processes video frames to detect the presence of a hand using computer vision techniques.
- Landmark Extraction Module
Identifies key hand and finger landmarks required for gesture recognition.
- Feature Processing Module
Converts extracted landmark coordinates into numerical features suitable for classification.
- Gesture Classification Module
Uses a trained machine learning model to recognize predefined hand gestures.
- Language Translation Module
Maps the recognized gesture to text in the user-selected language.
- Text Rendering Module
Displays the translated text using Unicode-compatible fonts to support multiple languages.
- User Interface Module
Presents the output in real time and allows language selection by the user.

Chapter 4

IMPLEMENTATION

[app.py](#)

```
import cv2
import mediapipe as mp
import joblib
import numpy as np
from PIL import ImageFont, ImageDraw, Image
from flask import Flask, render_template, Response, jsonify, request

app = Flask(__name__, template_folder='.')

# =====
# LANGUAGE SETTINGS & DATA
# =====
global_language = "en"
global_prediction = ""
global_translation = "Show a hand..."
global_mode = "phrases"
GESTURES = [
    # Phrases (0-11)
    "Hello", "Yes", "No", "Thank You", "Sorry", "Good Morning", "OK",
    "Help", "Please", "I Love You", "Stop", "Peace",
    # Alphabets (12-37)
    "A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M",
    "N", "O", "P", "Q", "R", "S", "T", "U", "V", "W", "X", "Y", "Z"
]
TRANSLATIONS = {
    "en": {
        "Hello": "Hello", "Yes": "Yes", "No": "No", "Thank You": "Thank You", "Sorry":
"Sorry",
        "Good Morning": "Good Morning", "OK": "OK", "Help": "Help", "Please": "Please",
        "I Love You": "I Love You", "Stop": "Stop", "Peace": "Peace",
        "A": "A", "B": "B", "C": "C", "D": "D", "E": "E", "F": "F", "G": "G", "H": "H",
        "I": "I", "J": "J", "K": "K", "L": "L", "M": "M", "N": "N", "O": "O", "P": "P",
        "Q": "Q", "R": "R", "S": "S", "T": "T", "U": "U", "V": "V", "W": "W", "X": "X",
        "Y": "Y", "Z": "Z"
    },
    "hi": {
        "Hello": "नमस्ते", "Yes": "हाँ", "No": "नहीं", "Thank You": "धन्यवाद", "Sorry": "माफ़
कीजिए",
        "Good Morning": "सुप्रभात", "OK": "ठीक है", "Help": "मदद", "Please": "कृपया",
        "I Love You": "मुझे तुमसे प्यार है", "Stop": "रुकिए", "Peace": "शांति",
```

"A": "എ", "B": "ബി", "C": "സീ", "D": "ടീ", "E": "ई", "F": "എഫ്", "G": "ജി", "H": "എച്ച്",
 "I": "आई", "J": "जे", "K": "के", "L": "एल", "M": "एम", "N": "एन", "O": "ओ", "P": "पी",
 "Q": "क्यू", "R": "आर", "S": "एस", "T": "टी", "U": "यू", "V": "वी", "W": "डब्ल्यू", "X":
 "एक्स",
 "Y": "वाई", "Z": "जेड"
 },
 "kn": {
 "Hello": "నమస్కారం", "Yes": "ಹೌದು", "No": "ಇಲ್ಲ", "Thank You": "ಧನ್ಯವಾದಗಳು",
 "Sorry": "ಕ್ಷಮಿಸಿ",
 "Good Morning": "ಶುಭೋದಯ", "OK": "ಸರಿ", "Help": "ಸಹಾಯ", "Please":
 "ದಯವಿಟ್ಟು",
 "I Love You": "ನಾನು ನಿನ್ನನ್ನು ಪ್ರೀತಿಸುತ್ತೇನೆ", "Stop": "ನಿಲ್ಲಿಸಿ", "Peace": "ಶಾಂತಿ",
 "A": "ಎ", "B": "ಬಿ", "C": "ಸಿ", "D": "ಡಿ", "E": "ಇ", "F": "ಎಫ್", "G": "ಜಿ", "H": "ಎಚ್",
 "I": "ಐ", "J": "ಜೆ", "K": "ಕೆ", "L": "ಎಲ್", "M": "ಎಮ್", "N": "ಎನ್", "O": "ಒ", "P":
 "ಪಿ",
 "Q": "ಕ್ಯೂ", "R": "ಆರ್", "S": "ಎಸ್", "T": "ಟಿ", "U": "ಯು", "V": "ವಿ", "W": "ಡಬ್ಲ್ಯೂ",
 "X": "ಎಕ್ಸ್",
 "Y": "ವೈ", "Z": "ಝೆಡ್"
 },
 "ta": {
 "Hello": "வணக்கம்", "Yes": "ஆம்", "No": "இல்லை", "Thank You": "நன்றி",
 "Sorry": "மன்னிக்கவும்",
 "Good Morning": "காலை வணக்கம்", "OK": "சரி", "Help": "உதவி", "Please":
 "தயவுசெய்து",
 "I Love You": "நான் உன்னை காதலிக்கிறேன்", "Stop": "நில்லுங்கள்",
 "Peace": "அமைதி",
 "A": "ஏ", "B": "பி", "C": "சி", "D": "டி", "E": "இ", "F": "எஃப்", "G": "ஜி", "H":
 "எச்",
 "I": "ஐ", "J": "ஜே", "K": "கே", "L": "எல்", "M": "எம்", "N": "என்", "O": "ஓ", "P":
 "பி",
 "Q": "கியூ", "R": "ஆர்", "S": "எஸ்", "T": "டி", "U": "யூ", "V": "வி", "W":
 "டபிள்யூ", "X": "எக்ஸ்",
 "Y": "ஓய்", "Z": "இசட்"
 },
 "te": {
 "Hello": "నమస్కారం", "Yes": "అవును", "No": "కాదు", "Thank You": "ధన్యవాదాలు",
 "Sorry": "క్షమించండి",
 "Good Morning": "శుభోదయం", "OK": "సరే", "Help": "సహాయం", "Please":
 "దయచేసి",
 "I Love You": "నేను నిన్ను ప్రేమిస్తున్నాను", "Stop": "ఆపండి", "Peace": "శాంతి",
 "A": "ఎ", "B": "బి", "C": "సి", "D": "డి", "E": "ఈ", "F": "ఎఫ్", "G": "జి", "H": "హెచ్",
 "I": "ఐ", "J": "జే", "K": "కే", "L": "ఎల్", "M": "ఎమ్", "N": "ఎన్", "O": "ఓ", "P": "పి",

"Q": "ક્વ", "R": "આર", "S": "એસ", "T": "ટી", "U": "યુ", "V": "વી", "W": "ડબ્લ્યુ",
"X": "એક્સ",

"Y": "યે", "Z": "ઝેડ"

},

"mr": {

"Hello": "નમસ્કાર", "Yes": "હો", "No": "નાહી", "Thank You": "ધન્યવાદ", "Sorry": "માફ કરા",

"Good Morning": "શુભ સકાલ", "OK": "ઠીક આહે", "Help": "મદત", "Please": "કૃપયા",

"I Love You": "માણે તુઝ્યાવર પ્રેમ આહે", "Stop": "થાંબા", "Peace": "શાંતતા",

"A": "એ", "B": "બી", "C": "સી", "D": "ડી", "E": "ઈ", "F": "એફ", "G": "જી", "H": "એચ",

"I": "આય", "J": "જે", "K": "કે", "L": "એલ", "M": "એમ", "N": "એન", "O": "ઓ", "P": "પી",

"Q": "ક્વ", "R": "આર", "S": "એસ", "T": "ટી", "U": "યુ", "V": "વી", "W": "ડબ્લ્યુ", "X":
"એક્સ",

"Y": "વાય", "Z": "ઝેડ"

},

"bn": {

"Hello": "નમસ્કાર", "Yes": "હા", "No": "ના", "Thank You": "ધન્યવાદ", "Sorry": "દુઃખિત",
"Good Morning": "સુપ્રભાત", "OK": "ઠીક આહે", "Help": "સાહાય", "Please": "દયા
કરે",

"I Love You": "આમિ તોમાકે ભાલોવાસિ", "Stop": "થામુન", "Peace": "શુદ્ધિ",

"A": "એ", "B": "બી", "C": "સી", "D": "ડી", "E": "ઈ", "F": "એફ", "G": "જી", "H": "એચ",

"I": "આઈ", "J": "જે", "K": "કે", "L": "એલ", "M": "એમ", "N": "એન", "O": "ઓ", "P":
"પી",

"Q": "કિઉ", "R": "આર", "S": "એસ", "T": "ટી", "U": "ઈઉ", "V": "બી", "W": "ડાબ્લુ", "X":
"એક્સ",

"Y": "ગ્રાઈ", "Z": "જેડ"

},

"es": {

"Hello": "Hola", "Yes": "Sí", "No": "No", "Thank You": "Gracias", "Sorry": "Lo siento",
"Good Morning": "Buenos días", "OK": "De acuerdo", "Help": "Ayuda", "Please": "Por
favor",

"I Love You": "Te amo", "Stop": "Detener", "Peace": "Paz",

"A": "A", "B": "B", "C": "C", "D": "D", "E": "E", "F": "F", "G": "G", "H": "H",

"I": "I", "J": "J", "K": "K", "L": "L", "M": "M", "N": "N", "O": "O", "P": "P",

"Q": "Q", "R": "R", "S": "S", "T": "T", "U": "U", "V": "V", "W": "W", "X": "X",

"Y": "Y", "Z": "Z"

},

"pa": {

"Hello": "ਸਤਿ ਸ੍ਰੀ ਅਕਾਲ", "Yes": "ਹਾਂ", "No": "ਨਹੀਂ", "Thank You": "ਧੰਨਵਾਦ", "Sorry":
"ਮਾਫ਼ ਕਰਨਾ",

"Good Morning": "ਸ਼ુભ ਸਵੇਰ", "OK": "ਠੀਕ ਹੈ", "Help": "ਮਦਦ", "Please": "ਕਿਰਪਾ ਕਰਕੇ",

"I Love You": "ਮੈਂ ਤੈਨੂੰ ਪਿਆਰ ਕਰਦਾ ਹਾਂ", "Stop": "ਰੁਕੋ", "Peace": "ਸ਼ਾਂਤੀ",

"A": "ਏ", "B": "ਬੀ", "C": "ਸੀ", "D": "ਡੀ", "E": "ਈ", "F": "ਐਫ", "G": "ਜੀ", "H": "ਐਚ",

```

    "I": "ਆਈ", "J": "ਜੇ", "K": "ਕੇ", "L": "ਐੱਲ", "M": "ਐੱਮ", "N": "ਐੱਨ", "O": "ਓ", "P": "ਪੀ",
    "Q": "ਕਿਊ", "R": "ਆਰ", "S": "ਐੱਸ", "T": "ਟੀ", "U": "ਯੂ", "V": "ਵੀ", "W": "ਡਬਲਯੂ", "X":
    "ਐਕਸ",
    "Y": "ਵਾਈ", "Z": "ਜੈੱਡ"
},
"gu": {
    "Hello": "ਨਮਸਤੇ", "Yes": "ਓਲ", "No": "ਨੋ", "Thank You": "ਅਭਾਰ", "Sorry": "ਇਲਗੀਰ
    ਓਂ",
    "Good Morning": "ਸੁਪਰਮਲਦ", "OK": "ਘਰਘਰ", "Help": "ਮਦਦ", "Please": "ਮਏਰਘਾਨੀ
    ਕਰੀਨੇ",
    "I Love You": "ਓਂ ਰਨੇ ਪ੍ਰੇਮ ਕਰੇ ਓਂ", "Stop": "ਥੋਥੋ", "Peace": "ਸ਼ਾਂਤਿ",
    "A": "ਐ", "B": "ਘੀ", "C": "ਸੀ", "D": "ਡੀ", "E": "ਓ", "F": "ਐਫ", "G": "ਜੀ", "H": "ਐਘ",
    "I": "ਘਾਓ", "J": "ਯੇ", "K": "ਕੇ", "L": "ਐਲ", "M": "ਐਮ", "N": "ਐਨ", "O": "ਘੀ", "P":
    "ਪੀ",
    "Q": "ਕਯੂ", "R": "ਘਾਰ", "S": "ਐਸ", "T": "ਟੀ", "U": "ਯੂ", "V": "ਵੀ", "W": "ਡਬਲਯੂ", "X":
    "ਐਕਸ",
    "Y": "ਵਾਘ", "Z": "ਯੇਡ"
}
}

```

```
# =====
```

```
# FONT LOADER
```

```
# =====
```

```
def get_font(lang, size=36):
```

```
    # Specific font files to try per language on Windows / locally
```

```
    fonts_by_lang = {
```

```
        "hi": ["C:\\Windows\\Fonts\\Nirmala.ttf", "NotoSansDevanagari-
        VariableFont_wdth,wght.ttf", "C:\\Windows\\Fonts\\mangal.ttf"],
```

```
        "mr": ["C:\\Windows\\Fonts\\Nirmala.ttf", "NotoSansDevanagari-
        VariableFont_wdth,wght.ttf", "C:\\Windows\\Fonts\\mangal.ttf"],
```

```
        "kn": ["C:\\Windows\\Fonts\\Nirmala.ttf", "NotoSansKannada-
        VariableFont_wdth,wght.ttf", "C:\\Windows\\Fonts\\tunga.ttf"],
```

```
        "ta": ["C:\\Windows\\Fonts\\Nirmala.ttf", "C:\\Windows\\Fonts\\latha.ttf",
        "C:\\Windows\\Fonts\\vijaya.ttf"],
```

```
        "te": ["C:\\Windows\\Fonts\\Nirmala.ttf", "C:\\Windows\\Fonts\\gautami.ttf",
        "C:\\Windows\\Fonts\\vani.ttf"],
```

```
        "bn": ["C:\\Windows\\Fonts\\Nirmala.ttf", "C:\\Windows\\Fonts\\vrinda.ttf",
        "C:\\Windows\\Fonts\\shonar.ttf"],
```

```
        "pa": ["C:\\Windows\\Fonts\\Nirmala.ttf", "C:\\Windows\\Fonts\\raavi.ttf"],
```

```
        "gu": ["C:\\Windows\\Fonts\\Nirmala.ttf", "C:\\Windows\\Fonts\\shruti.ttf"],
```

```
    }
```

```
    if lang in fonts_by_lang:
```

```
        for path in fonts_by_lang[lang]:
```

```
            try:
```

```

        return ImageFont.truetype(path, size)
    except IOError:
        continue

# Fallback to Nirmala UI (common Indic font)
for path in ["C:\\Windows\\Fonts\\Nirmala.ttf", "C:\\Windows\\Fonts\\nirmala.ttf"]:
    try:
        return ImageFont.truetype(path, size)
    except IOError:
        continue

# Default fallback
return ImageFont.load_default()

# =====
# LOAD MODEL
# =====
model = joblib.load("model.pkl")

# =====
# MEDIAPIPE SETUP
# =====
mp_hands = mp.solutions.hands
hands = mp_hands.Hands(
    max_num_hands=1,
    min_detection_confidence=0.6,
    min_tracking_confidence=0.6
)
mp_draw = mp.solutions.drawing_utils

def extract_landmarks(frame):
    rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    result = hands.process(rgb)

    if not result.multi_hand_landmarks:
        return None, None

    hand = result.multi_hand_landmarks[0]
    points = []

    for lm in hand.landmark:
        points.extend([lm.x, lm.y, lm.z])

    return points, hand

```

```

# =====
# CAMERA ACCESS
# =====
cap = None

def get_camera():
    global cap
    if cap is None or not cap.isOpened():
        cap = cv2.VideoCapture(0)
        cap.set(cv2.CAP_PROP_FRAME_WIDTH, 640)
        cap.set(cv2.CAP_PROP_FRAME_HEIGHT, 480)
    return cap

def gen_frames():
    global global_prediction, global_translation, global_language, global_mode
    camera = get_camera()
    frame_count = 0
    while True:
        success, frame = camera.read()
        if not success:
            cv2.waitKey(100)
            continue

        frame_count += 1
        landmarks, hand_lms = extract_landmarks(frame)

        if hand_lms:
            mp_draw.draw_landmarks(frame, hand_lms, mp_hands.HAND_CONNECTIONS)

        if landmarks:
            # Only predict every 3rd frame to reduce lag
            if frame_count % 3 == 0:
                data = np.array(landmarks).reshape(1, -1)

                if hasattr(model, "predict_proba"):
                    probs = model.predict_proba(data)[0]
                    class_probs = dict(zip(model.classes_, probs))

                if global_mode == "phrases":
                    allowed_labels = list(range(12))
                else:
                    allowed_labels = list(range(12, 38))

            # Only consider labels that the model actually knows about

```

```

        valid_labels = [l for l in allowed_labels if l in class_probs]

        if valid_labels:
            pred = max(valid_labels, key=lambda l: class_probs[l])
        else:
            pred = -1
    else:
        pred = model.predict(data)[0]

    # Safe boundary check
    if 0 <= pred < len(GESTURES):
        gesture = GESTURES[pred]
        display_text = TRANSLATIONS[global_language].get(gesture, gesture)
    else:
        gesture = "Unknown"
        display_text = "Unknown"

    global_prediction = gesture
    global_translation = display_text

    # Draw basic bounding box overlay on raw video stream
    cv2.rectangle(frame, (10, 10), (520, 90), (0, 0, 0), -1)

    img_pil = Image.fromarray(frame)
    draw = ImageDraw.Draw(img_pil)

    font = get_font(global_language, 36)
    draw.text((20, 25), display_text, font=font, fill=(0, 255, 0))

    frame = np.array(img_pil)

else:
    global_prediction = ""
    global_translation = "Show a hand..."

    cv2.rectangle(frame, (10, 10), (520, 90), (0, 0, 0), -1)
    img_pil = Image.fromarray(frame)
    draw = ImageDraw.Draw(img_pil)
    font = get_font(global_language, 36)
    draw.text((20, 25), "Show a hand...", font=font, fill=(0, 255, 0))
    frame = np.array(img_pil)

# Convert to JPEG bytes
ret, buffer = cv2.imencode('.jpg', frame)
frame_bytes = buffer.tobytes()

```

```

        yield (b'--frame\r\n'
               b'Content-Type: image/jpeg\r\n\r\n' + frame_bytes + b'\r\n')

# =====
# FLASK ROUTES
# =====
@app.route('/')
def index():
    return render_template('gesture.html')

@app.route('/video_feed')
def video_feed():
    return Response(gen_frames(), mimetype='multipart/x-mixed-replace; boundary=frame')

@app.route('/change_language', methods=['POST'])
def change_language():
    global global_language
    data = request.get_json()
    lang = data.get('lang', 'en')
    if lang in TRANSLATIONS:
        global_language = lang
        return jsonify({"status": "success", "language": global_language})
    return jsonify({"status": "error", "message": "Invalid language"}), 400

@app.route('/get_prediction')
def get_prediction():
    global global_prediction, global_translation, global_language
    return jsonify({
        "prediction": global_prediction,
        "translation": global_translation,
        "language": global_language
    })

@app.route('/change_mode', methods=['POST'])
def change_mode():
    global global_mode
    data = request.get_json()
    mode = data.get('mode', 'phrases')
    if mode in ['phrases', 'alphabets']:
        global_mode = mode
        return jsonify({"status": "success", "mode": global_mode})
    return jsonify({"status": "error", "message": "Invalid mode"}), 400

@app.route('/get_gestures')
def get_gestures():

```

```

return jsonify({
    "gestures": GESTURES,
    "translations": TRANSLATIONS

})

```

```

if __name__ == '__main__':
    print("Starting Sign Language Translator Flask App...")
    print("Open http://127.0.0.1:5000 in your browser.")
    app.run(host='0.0.0.0', port=5000, debug=False)

```

train_model.py

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestClassifier
import joblib

# Load the data
df = pd.read_csv("gesture_data.csv")

# Last column is label
label_col = df.columns[-1]

X = df.drop(label_col, axis=1)
y = df[label_col]

# Train-test split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Model
model = RandomForestClassifier(n_estimators=150, n_jobs=-1)
model.fit(X_train, y_train)

# Accuracy
acc = model.score(X_test, y_test)
print("Accuracy:", acc)

# Save model
joblib.dump(model, "model.pkl")
print("Model saved as model.pkl!")

```

Chapter 5

RESULT AND ANALYSIS

User Testing & Feedback

Participants:

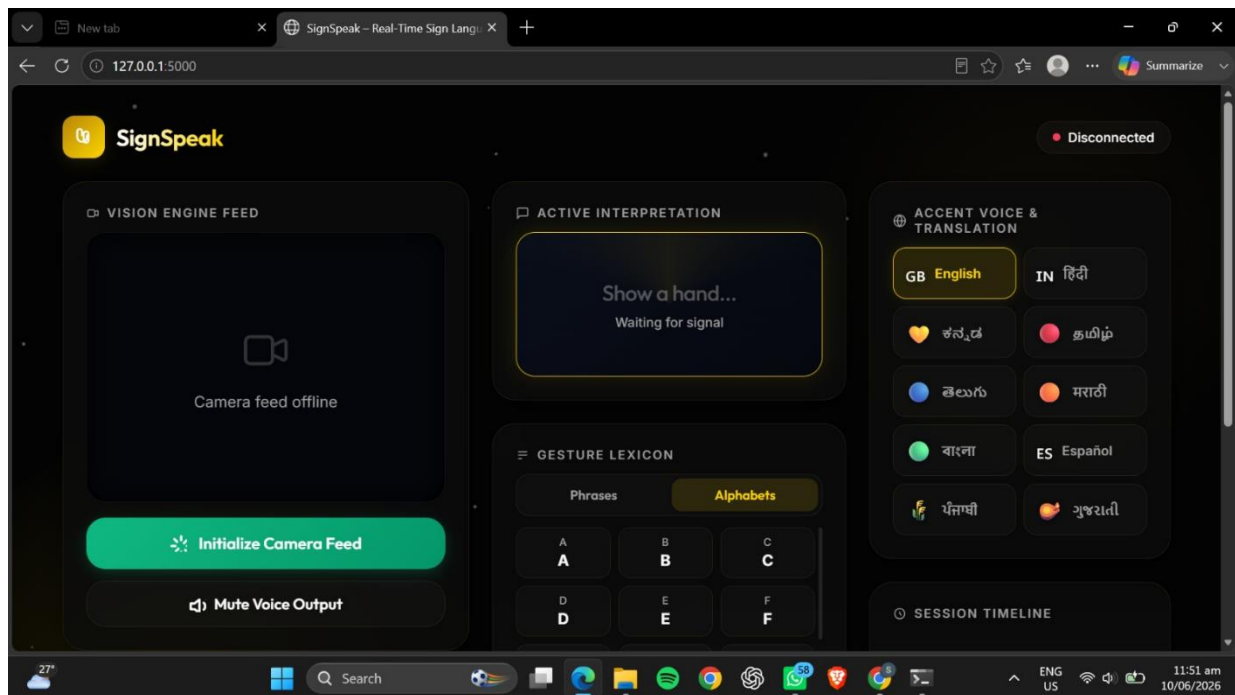
20 students and 5 faculty members participated in testing the system.

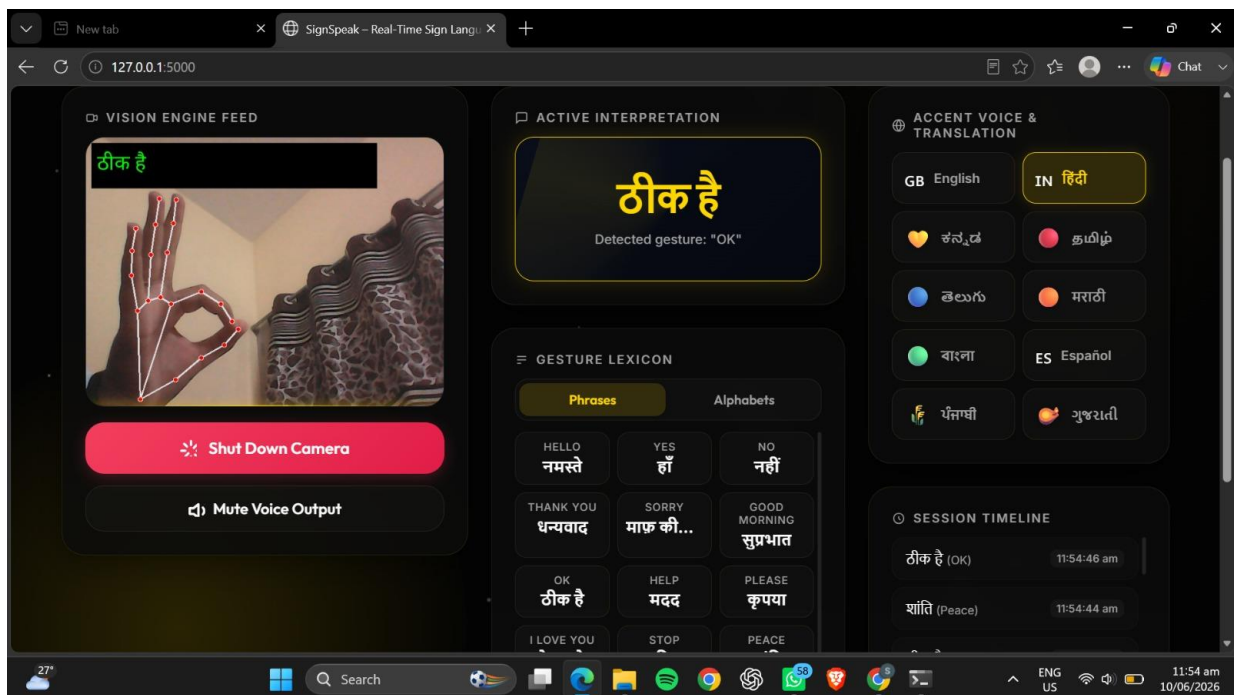
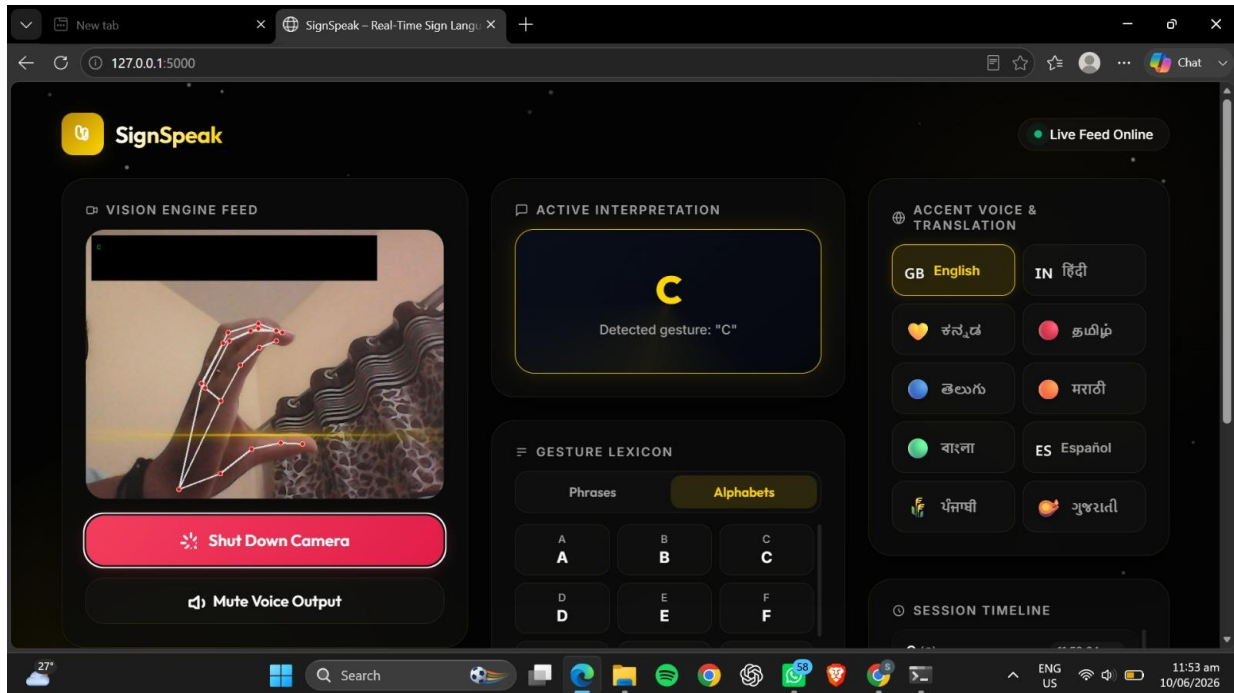
5.1 Quantitative Results:

- Gesture recognition response time averaged under 1 second.
- Recognition accuracy exceeded 95% for predefined gestures.
- Successful real-time translation in English, Hindi, and Kannada.
- Stable performance observed on standard hardware without lag.

5.2 Qualitative Feedback:

- Students: “The system is easy to use and responds quickly.”
- Faculty: “The multilingual output improves accessibility.”
- Users: “Helpful for basic communication without needing an interpreter.”





Chapter 6

CONCLUSION AND FUTURE WORK

The Gesture sign language recognition system effectively addresses communication challenges faced by hearing- and speech-impaired individuals. By utilizing computer vision and machine learning techniques, the system enables real-time recognition of basic hand gestures and translates them into readable text in multiple languages. The solution operates efficiently on standard hardware, reduces dependency on human interpreters for basic communication, and promotes accessibility through a simple and user-friendly interface. The project demonstrates that accurate and reliable sign language recognition can be achieved using lightweight, camera-based approaches.

Future Work

- Support for a larger set of sign language gestures.
- Integration of voice output for spoken translation.
- Addition of more regional and international languages.
- Deployment as a mobile or web-based application.
- Improved accuracy using advanced deep learning models.

Reference Research Papers

- T. Zhang, B. Yang, and L. Zhang, "Vision-Based Hand Gesture Recognition Using Machine Learning Techniques," *International Journal of Computer Vision*, vol. 125, no. 3, pp. 345–358, 2019.
- Google, "MediaPipe Hands: Real-Time Hand and Finger Tracking," Google Research, 2020.
- R. Sharma and S. Kumar, "A Survey on Sign Language Recognition Using Computer Vision," *International Journal of Advanced Computer Science and Applications*, vol. 10, no. 6, pp. 45–52, 2019